

CMPS 102 — Spring 2020 – Homework 2

Updated Ver. 1(14-Oct)

This assignment comprises of six questions and is worth fifty points. It is due on October 26 (10 a.m.) on Gradescope.

Before you begin the assignment, please read the following carefully.

- Read the *Homework Guidelines*.
- Every part of each question begins on a new page. Do not change this.
- This does not mean that you should write a full page for every question. Your answers should be short and precise. Lengthy and wordy answers will lose points.
- Do not change the format of this document. Simply type your answers as directed.
- You are **not** allowed to work in teams.

I have read and agree to the collaboration policy. – FirstName LastName, email@ucsc.edu
Collaborators:

1. (Total: 8 pts) A grad student comes up with the following algorithm to sort an array $A[1..n]$ that works by first sorting the first 2/3rds of the array, then sorting the last 2/3rds of the (resulting) array, and finally sorting the first 2/3rds of the new array.

```
1: function G-SORT( $A, n$ )                                ▷ takes as input an array of  $n$  numbers,  $A[1..n]$ 
2:   G-sort-recurse( $A, 1, n$ )
3: end function
4: function G-SORT-RECURSE( $A, \ell, u$ )
5:   if  $u - \ell \leq 0$  then
6:     return                                             ▷ 1 or fewer elements already sorted
7:   else if  $u - \ell = 1$  then                             ▷ 2 elements
8:     if  $A[u] < A[\ell]$  then                               ▷ swap values
9:        $\text{temp} \leftarrow A[u]$ 
10:       $A[u] \leftarrow A[\ell]$ 
11:       $A[\ell] \leftarrow \text{temp}$ 
12:   end if
13: else                                                     ▷ 3 or more elements
14:    $\text{size} \leftarrow u - \ell + 1$ 
15:    $\text{twothirds} \leftarrow \lceil (2 * \text{size}) / 3 \rceil$ 
16:   G-sort-recurse( $A, \ell, \ell + \text{twothirds} - 1$ )
17:   G-sort-recurse( $A, u - \text{twothirds} + 1, u$ )
18:   G-sort-recurse( $A, \ell, \ell + \text{twothirds} - 1$ )
19: end if
20: end function
```

a (4 pts). First, prove that the algorithm correctly sorts the numbers in the array (in increasing order). After showing that it correctly sorts 1 and 2 element intervals, you may make the (incorrect) assumption that the number of elements being passed to *G-sort-recurse* is always a multiple of 3 to simplify the notation (and drop the floors/ceilings).

Solution.

Base cases:

- (a) $u - l = 0$ (one element): Test on line 5 is true.
 $\hookrightarrow$ algorithm returns the array with no change
Sort: If length of the array is 1, we shouldn't do anything.
- (b) $u - l = 2$: Test on line 5 is false, Test on line 7 is true and $8 \Rightarrow A[u]$ and $A[l]$ are arranged in decreasing order
 $\hookrightarrow$ Line 8 evaluates to true and the algorithm returns the array of size 2 swapped.

Case $n \% 3 == 0$:

$$Ex.u = 12, l = 4$$

$$u - l = 8$$

$$size = 8 + 1 = 9$$

$$twothirds = (2 * 9) / 3 = 6$$

- (a) $G\text{-sort-recurse}(A, 4, 4+6-1) \Rightarrow G\text{-sort-recurse}(A, 4, 9)$
- (b) $G\text{-sort-recurse}(A, 12-6+1, 12) \Rightarrow G\text{-sort-recurse}(A, 7, 12)$
- (c) $G\text{-sort-recurse}(A, 4, 4+6-1) \Rightarrow G\text{-sort-recurse}(A, 4, 9)$

$A[7 \dots 9]$ is sorted twice.

IH: $n > 2$ and $n \% 3 == 0$

Test on line 5 and line 7 is false therefore lines 14-18 will execute

By IH, the calls *G-sort-recurse*() with sub-arrays whose length is j size are correct. From this the corresponding sub-arrays will eventually get sorted when $m_j=2$. The sub-array $A[l \dots u]$ is sorted in these sections:

$$X = A[l \dots l + twothirds - 1]$$

$$Y = A[u - twothirds + 1 \dots u]$$

$$Z = A[l \dots l + twothirds - 1]$$

After line 16 is executed sub-arrays X and Y are sorted

$\hookrightarrow X \leq Y$ since every element in X is less than Y

After line 17 is executed sub-arrays Y and Z are sorted, $Y \leq Z$

$X \leq Y$ and $Y \leq Z \Rightarrow X \leq Z$ *not necessarily*

Sub-arrays X and Z are sorted again in line 18 so that $X \leq Z$ holds true.

X, Y, Z are then truly sorted

b (1 pts). Next, Derive a recurrence for the algorithm's running time (or number of comparisons made).

Solution. *There are 3 recursive calls each partitioning off the array into a sub array that is $\frac{2}{3}$ the size of the original array. The sub arrays are either left or right aligned. Therefore the resulting piecewise function should look like this:*

$$T(n) = \begin{cases} 1 & n \leq 2 \\ 3 \cdot T(\frac{2n}{3}) + 1 & n \geq 3 \ \&\& \ n \% 3 = 0 \end{cases}$$

c (3 pts). Finally, obtain a good asymptotic upper bound (big- O) for your recurrence.

Solution.

MT:

$$T(n) = 3 \cdot T\left(\frac{2n}{3}\right) + 1$$

In the form $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$

$$a = 3$$

$$b = \frac{3}{2}$$

$$f(n) = 1 = n^0$$

Now we compare $n^{\log_b a}$ and n^0

$$\log_b a = \log_{\frac{3}{2}} 3 = \frac{\log 3}{\log \frac{3}{2}} \approx 2.71$$

$$n^{\log_b a} \approx n^{2.71} \text{ vs } n^0 \Rightarrow n^{2.71} > n^0$$

since

$$n^{\log_b a} > n^0$$

we can move forward with Case 1:

"Look" for ϵ :

$$\epsilon = \log_{\frac{3}{2}} 3, \epsilon > 0$$

$$O(n^0) = O(n^{\log_{\frac{3}{2}} 3 - \epsilon})$$

By Case 1:

$$T(n) = \Theta(n^{\log_{\frac{3}{2}} 3}) = \Theta(n^{2.71})$$

2. (Total: 3 pts) Least counterexample: Give a proof by contradiction using the *least counterexample* method that:

$$\sum_{i=0}^n i = \frac{n(n+1)}{2} \quad \text{for all } n \geq 0.$$

Solution.

Assume $\sum_{i=0}^n i \neq \frac{n(n+1)}{2}$

$$\sum_{i=0}^n i = n + \sum_{i=0}^{n-1} i$$

$$\sum_{i=0}^{n-1} i = 1 + 2 + 3 + 4 + \dots + (n-1) = \frac{n^2 - n}{2}$$

$$n + \sum_{i=0}^{n-1} i = n + \frac{n^2 - n}{2}$$

$$n + \frac{n^2 - n}{2} = \frac{n^2 - n + 2n}{2} = \frac{n^2 + n}{2} = \frac{n(n+1)}{2}$$

$$\sum_{i=0}^n i = \frac{n(n+1)}{2}$$

3. (Total: 8 pts) Define the sequence $T_n = (n-1) + \frac{n-1}{n^2} \cdot \sum_{k=1}^{n-1} T_k$. Prove, by induction, for all $n \geq 1$, that $T_n \leq 2n$.

Solution.

Base case:

$$T_1 = ((1) - 1) + \frac{(1) - 1}{(1)^2} \cdot \sum_{k=1}^{(1)-1} T_k = 0 + 0 \cdot \sum_{k=1}^0 T_k = 0 \leq 2n \quad \checkmark$$

Assume:

$$T_n \leq 2n$$

Then:

$$T_{n+1} = ((n+1) - 1) + \frac{(n+1) - 1}{(n+1)^2} \cdot \sum_{k=1}^{(n+1)-1} T_k = (n) + \frac{n}{(n+1)^2} \cdot \sum_{k=1}^n T_k$$

$$\sum_{k=1}^n T_k = \sum_{k=1}^n 2k \quad \text{since we assumed} \quad T_k \leq 2k$$

$$= 2 \cdot \sum_{k=1}^n k = 2 \cdot \frac{n(n+1)}{2}$$

$$(n) + \frac{n}{(n+1)^2} \cdot \sum_{k=1}^n T_k = (n) + \frac{n}{(n+1)^2} \cdot 2 \cdot \frac{n(n+1)}{2}$$

$$(n) + \frac{n}{(n+1)^2} \cdot 2 \cdot \frac{n(n+1)}{2} = 2n + \frac{1}{n+1} - 1$$

Now:

$$2n + \frac{1}{n+1} - 1 \leq 2n$$

$$\frac{1}{n+1} - 1 \leq 0$$

$$\frac{1}{n+1} \leq 1$$

If we substitute the least value n can be ($n \geq 1$) then the statement above ($\frac{1}{n+1} \leq 1$) holds true.

4. (Total: 14 pts) Let $S(n)$ be defined, for $n \geq 2$, as $S(n) = 2S(\lceil \frac{n}{2} \rceil) + n \log_2(n)$. For $n = 1$, the value is $S(n) = 1$.

- (a) (4 pts) Can you apply Master Theorem on $S(n)$? If no, precisely state why not. If yes, apply the theorem and derive the bound.

Solution.

MT:

In the form $T(n) = a \cdot T(\frac{n}{b}) + f(n)$

$$a = 2$$

$$b = 2$$

$$f(n) = n^c \log^k n = n \log_2(n) \Rightarrow \text{Degree} = 1, k = 1$$

Now we compare $n^{\log_b a}$ and $n \log_2(n)$

$$\log_b a = \log_2 2 = 1$$

To compare:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{n^{\log_b a - \epsilon}} = \lim_{n \rightarrow \infty} \frac{n \log_2(n)}{n^{1-\epsilon}} = \lim_{n \rightarrow \infty} \frac{\log_2(n)}{n^\epsilon} \Rightarrow \infty$$

This does not fit for Case 2: we say that there is a non polynomial difference since the Limit calculated is infinity. Both $f(n)$ and $n^{\log_b a}$ are at the degree of 1 but they are not polynomially the same. If they were polynomially the same then the Limit calculated earlier will equal to 1 but there is a non polynomial difference that ejects this from any of the 3 master's theorem cases.

(b) (10 pts) Now use a method, besides Master Theorem, to prove that $S(n) \in O(n(\log_2 n)^2)$.

Solution.

$$S(n) = 2S(\lceil \frac{n}{2} \rceil) + n \log_2(n)$$

Let's say $n = 2^m$:

$$S(2^m) = 2S(\lceil \frac{2^m}{2} \rceil) + 2^m \log_2(2^m) = 2S(2^{m-1}) + m2^m$$

If we substitute $S(2^m)$ with $f(m)$:

$$\begin{aligned} f(m) &= 2f(m-1) + m2^m \\ &= 2(2f(m-2) + (m-1)2^{m-1}) + m2^m \\ &= 4f(m-2) + (m-1)2^m + m2^m \\ &= 2(4f(m-3) + (m-2)2^{m-2}) + (m-1)2^m + m2^m \\ &= 8f(m-3) + (m-2)2^m + (m-1)2^m + m2^m \\ &\quad \vdots \\ f(m) &= 2^m f(0) + 2^m \left(\sum_{j=1}^m j \right) \\ &= 2^m f(0) + 2^m \cdot \frac{m(m+1)}{2} \\ &= 2^m f(0) + 2^{m-1} \cdot m(m+1) \end{aligned}$$

From this we say:

$$\begin{aligned} S(n) &= nS(1) + n \left(\frac{(\log_2(n)(1 + \log_2(n)))}{2} \right) \\ &= n + n \left(\frac{(\log_2(n) + (\log_2 n)^2)}{2} \right) \\ &= n + \left(\frac{n \log_2(n) + n(\log_2 n)^2}{2} \right) \\ &= n + \left(\frac{n \log_2(n)}{2} \right) + \left(\frac{n(\log_2 n)^2}{2} \right) \end{aligned}$$

Highest degree $\Rightarrow n(\log_2 n)^2$:

$$\Theta(n(\log_2 n)^2)$$

5. (5 points) You are presented with two sorting algorithms. You have to select one of them. The documentation provided states that both of these algorithms perform $\Omega(n^3)$ comparisons in the worst case. You want to build a fast program and therefore want to optimise speed. Does it matter which algorithm you select? How would you make the decision?

Solution. *To decide on the algorithm to use, One must look at the big O complexity if available. If not then look at the runtime of the algorithms and the memory used during runtime. The less memory used the more efficient. Both have a best case n^3 of comparisons so regardless of the n you pass to the algorithm both will perform n^3 at best. But since the algorithms are stated to have $\Omega(n^3)$ in worst case that implies $O(n^3)$. Unless that is a typo, that line indicates that either algorithm is fine to use either algorithm since the best case (Ω) and worst case (O) are the same then the memory use and runtime speed are the factors to consider.*

6. (Total: 12 pts) You are interested in building a coffeeshop and charging station beside a freeway running between two cities. Let n be the number of interchanges between the cities, and number them consecutively so that interchange 0 is at the first city, 1 is the next interchange, and so on, with $n - 1$ being the interchange at the other city. We say each interchange i is adjacent to interchanges $i - 1$ and $i + 1$ (only).

Each interchange i has a cost $c(i)$ to construct a coffee shop/charging station. Call an interchange i a *local minimum* if (and only if) the cost of building there is less than the costs of building at the adjacent interchanges, i.e. both $c(i) < c(i + 1)$ and $c(i) < c(i - 1)$. You want to ensure that you build at a local minimum so that no one can build a coffeeshop/charging station more cheaply at an adjacent interchange.

You can assume that:

- $n \geq 3$, so there is at least one interchange between the cities.
- The costs at the cities $c(0)$ and $c(n - 1)$ are both known to be the same large number (say $100n$).
- The other costs are all different and are all strictly less than $c(0)$ (and thus also less than $c(n - 1)$).

The difficulty is that the $c(i)$ values are not known ahead of time (except for $c(0)$ and $c(n - 1)$) and it is expensive to determine the cost of building at any interchange.

The goal is to create a divide and conquer algorithm that finds any interchange k that is a local minimum while examining relatively few of the $c(i)$ values. Although there may be many local minimums, your algorithm need only find one of them.

- (a) (2 points) First, prove that a local minimum exists.

Solution.

Given $c(0)$ and $c(n-1)$ of which we know to be identical local maximums. the cost of any point between interchange 0 and interchange $n-1$ is stricly lower. From this we can say that any point in between 0 and $n-1$ can be a minimum. Now to prove that it is a local minimum:

We can use the Extreme value theorem. Normally it is used for functions and in this case the function can be represented by $c(x)|x \in [0, n - 1]$ The extreme value theorem states that within a continuous function there shall be an absolute maximum and minimum attained at least once. We know that the index 0 and index $n-1$ are local maximums and are greater than the rest of the interchange cost values therefore they are absolute maximums. Following that logic there is also an absolute minimum within the interval $[0, n - 1]$. We can say that the function is continuous because there is a cost value at every interchange and there are no breaks in value.

Suppose the function f which represents the interchanges and is continuous with every interchange having a value that is less than the first and last interchange. f at one point will reach an absolute minimum in the interval $[0, n - 1]$. Becasue we have a local maximum at 0 and $n-1$, there is an interval $[0, 0 + \delta)$ such that $f(x_1) \geq f(x)$ for all x in $[0, x_1 + \delta)$.

(b) (5 points) Clearly describe a divide and conquer algorithm for the problem.

Two pages have been left blank for this question. If you do not need both the page, please leave the second page blank.

```
1         function locMin(arr, lo, hi, n) {
2             mid = floor((lo+hi-lo)/2)
3             if mid==0 || arr[mid-1] > arr[mid] && mid==n-1
4                 || arr[mid+1] > arr[mid] {
5                 return mid
6             } else if mid>0 && arr[mid-1] < arr[mid] {
7                 return locMin(arr, lo, mid-1, n)
8             } else {
9                 return locMin(arr, mid+1, hi, n)
10            }
        }
```

Solution.

Assumptions:

*input is an array of size ≥ 3 containing the costs for each index which represents an interchange,
lo = 0, hi = n-1, n = size of array*

Algorithm will look at the middle index then check it to see if its a local minima Else recurse with a respective subarray and check the middle of that. Essentially binary search with more conditions than just equality

(c) (3 points) Prove that your algorithm is correct.

Solution.

Base Case:

Where $n \geq 3$ we know that mid will be a local minima since index 0 and index $n-1$ are strictly greater than anything in between.

IH:

Case localMin at mid: Function tests true at line 3

$\hookrightarrow$ returns interchange number represented by the index

Case not localMin at mid & $mid-1 < mid$: Function tests false at line 3 and true at line 5

$\hookrightarrow$ returns recursion of the lower half sub-array

This will recurse with a sub-array with values that are less than mid and lo. This will hold true because it is narrowing down to the minima since there are values lesser than mid and lo.

Case not localMin at mid & $mid+1 < mid$: Function tests false at line 3 and false at line 5

$\hookrightarrow$ returns recursion of the upper half sub-array

This will recurse with a sub-array with values that are less than mid and hi. This will hold true because it is narrowing down to the minima since there are values lesser than mid and hi.

- (d) (2 points) Analyze the (worst case) number of interchange costs (i.e. $c(i)$ values) examined by your algorithm as a function of n .

Solution.

There is only 1 recursion function at any given time and n is assumed to be the array/sub-array passed in

$$T(n) = \begin{cases} 1 & \text{if local Minima at mid or first or last interchange and } n = 3 \\ T(\frac{n}{2}) + 1 & \text{if not local Minima or first or last interchange and } n > 3 \end{cases}$$

MT:

$$T(n) = T(\frac{n}{2}) + 1$$

In the form $T(n) = a \cdot T(\frac{n}{b}) + f(n)$

$$a = 1$$

$$b = 2$$

$$f(n) = 1 = n^0 \Rightarrow c = 1$$

Now we compare $n^{\log_b a}$ and n^0

$$\log_b a = \log_2 1 = 0$$

$$n^{\log_b a} = n^0 \text{ vs } n^0 \Rightarrow n^0 = n^0$$

since

$$n^{\log_b a} = n^0$$

we can move forward with Case 2:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(n^1 \cdot \log n) = \Theta(n \cdot \log n)$$

The algorithm is based around Binary search whose time complexity is the same as above $\Theta(n \cdot \log n)$